MLOps Engineering

Machine Learning Operations V4.0.0

1st Group Assignment

MsC in Business Analytics and Data Science

Madrid, Feb 2026

ie UNIVERSITY

Expectations for both best practices to be showcased, and the modules to be delivered for this first phase

General best practices

1. **Modularization**: Break down code into smaller, reusable functions or classes
2. **Dependency Management**: List all dependencies and versions for reproducibility
3. **Error Handling**: Include comprehensive error handling and data validation
4. **Testing**: Include unit tests to validate the functionality
5. **Documentation**: Include sufficient comments and documentation for maintainability
6. **Code Quality**: Ensure the code is clean, readable, and follows PEP 8 standards
7. **Configuration Management**: Use configuration files or MLOps 101ment variables for settings
8. **Logging and Monitoring**: Add logging and possibly some basic monitoring metrics
9. **Security**: Remove any sensitive or secret info (.env)

ML Pipeline specific

1. **data_loader.py**: Load data from CSV, databases, or APIs
2. **data_cleaner**: Ensures the raw data is compliant so the pipeline behaves predictably
3. **data_validation.py**: Validate schema, check types, handle missing data
4. **preprocessing.py**: Clean, scale, encode, and transform input data
5. **features.py**: Create and select engineered features
6. **train.py**: Define, train, save, and load ML models
7. **evaluation.py**: Compute and log performance metrics
8. **inference.py:** Apply trained model to new data (predict pipeline)
9. **main.py**: Orchestrate the full ML pipeline for traceability and robust error handling
10. **utils.py** (optional): Shared utilities (e.g. logging setup, plotting functions)
11. **config.yaml**: Centralize parameters, paths, and MLOps 101ment configs

Good code isn't just easier to read, it's essential for a successful ML project

Coding best practices

- Writing clean and modular code
- Refactoring code
- Optimizing code for efficiency
- Writing documentation
- Following PEP8 guidelines and linting

Production ready

- Unit test and I/O data validation
- Logging and tracking
- Command line arguments

GitHub collaboration

- Review of the add, commit, and push
- Branching
- Common version control workflows
- Code reviews

Source: Based on Udacity framework

1st Group assignment expectations and guidelines

Category	Details
Code Quality	• The code is following the **PEP8 coding style** (No errors on VSC > PROBLEMS tab) e.g. Meaningful names and syntax; properly commented and formatted; Imports are ordered, etc.
Documentation	• There's a comprehensive **README.rm** providing a general (preliminary) overview of the project, instructions on how to use the code, run tests, etc. [Anyone should be able to run the code from those instructions] • All functions and files have **docstrings**, and **commenting** is used appropriately i.e. **Why NOT What**
Testing	• **Tests cover most functions** and relevant artifacts e.g. Inputs are in the expected file format? Outputs are saved in the right places? Edge cases? (pytest-cov)
Dependencies	• **Dependencies** are managed through a **MLOps 101ment.yml** and/or **conda.yml** file, listing all dependencies and versions for reproducibility
Error handling	• Error Handling thru **try/ except/ raise** provides a comprehensive error handling and **data type validation**
Artifacting	• Relevant reports, notebooks, or images of plots from the EDA phase are stored on a respective folder • A model(s) is properly stored (joblib or pickle) for future production use in inference phases
ML Pipeline Modules	The code modules cover and end-to-end ML process e.g. • data_loader.py: Load data from CSV, databases, or APIs • data_cleaner: Ensures the raw data is compliant so the pipeline behaves predictably • data_validation.py: Validate schema, check types, handle missing data • preprocessing.py: Clean, scale, encode, and transform input data • features.py: Create and select engineered features • train.py: Define, train, save, and load ML models • evaluation.py: Compute and log performance metrics • inference.py: Apply trained model to new data (predict pipeline) • main.py: Orchestrate the full ML pipeline for traceability and robust error handling • utils.py (optional): Shared utilities (e.g. logging setup, plotting functions)
Version Controlling	• GitHub is used for (basic) **Version Control and development,** demonstrating: - Follow best practices (as discussed in class) for: 1. Project creation 2. Branching (Check out features, debugging, development vs. main) 3. Pulls Request reviewing (Open, review and merge) 4. Main and Dev are properly protected

Overall project expectations and guidelines (Tech Section)

Criterion	Fundamentals missing (0–4)	Basic attempt (5–6)	Good implementation (7–8)	Excellent – Industry grade (9–10)
Single Entry Point & Modularization	Workflow poorly extracted from notebooks; minimal modularization	Basic script exists, limited/ not complete modularization, manual intervention required	Main script runs most modules; partial orchestration	Fully orchestrated modular pipeline
Code Quality & Efficiency	Frequent violations of best practices (PEP 8), inefficient code	Occasional issues; some inefficient loops or duplication	Generally clean, minimal duplication; minor efficiency gaps	Zero PEP 8 issues, fully optimized, vectorized, and efficient code
Documentation & Clarity	Missing/incomplete README, no docstrings	README exists but minimal; incomplete docstrings, comments miss the "Why?"	Useful README; most functions documented, further clarity improvements needed	Comprehensive README; all functions have clear docstrings, and relevant comments (Why)
Testing & Coverage	Minimal tests (<20% coverage), no edge cases	Basic tests (~50% coverage), lacks critical edge cases	Good tests (~75% coverage), some edge cases covered	Extensive tests (≥90% coverage), thorough edge-case tests and mock-up data
Dependency Management	Hard-coded paths, no MLOps 101ment management	Basic use of configuration (SETTINGS) or dependencies managed	Proper dependency management, partial config usage	Fully configurable, reproducible environments
Error Handling & Validation	Frequent uncaught exceptions, no validation	Basic try/except blocks; insufficient custom error handling	Adequate error handling; some schema validation implemented	Comprehensive error handling with custom exceptions, full validation
Artifacting & Reproducibility	No artifacts or outputs systematically stored	Partial storage; unclear or inconsistent model artifact paths	Clear storage for models; some reports or images documented	Fully reproducible artifacts; clearly structured assets
Pipeline Completeness (main & notebook)	Partial implementation (≤4 core modules)	Majority implemented (≥5 core modules); loosely coupled	All core modules implemented; some coupling issues	Complete modular pipeline; seamless integration
Version Control & Workflow	Minimal GitHub use; unclear or no evidence of individual contributions	Basic GitHub use; partial evidence of contributions from some members	Good GitHub practices; clear but uneven distribution of contributions	Strong GitHub use; explicit evidence each member owns distinct code/tests modules, showing significant individual contributions

(Preliminary) Basic Business Section guide on README.md

Client
– Client name and industry

Business Unit
- Business Unit or department

What's the maturity of the client
– Data, Tools, Processes, People, Strategy

Goal of Project (Objective metric, Improvement over baseline)
– In business or client terms
– **As measured by (quantifiable KPI)?**

Problem Statement
– What's the key pain point for the client?
– State the business pain point and current baseline metric
– **As measured by (quantifiable KPI)?**

Solution Description & Key Functionalities
- Simple, executive level description of the solution (Non-technical terms)

Solution Scalability
– How can this solution be applied to other use cases, industries, business functions?
– Can the solution grow in performance and scope, how?

Client Benefit (Over non-AI approach)
– Tangible (or intangible benefits for the client that wouldn't be attainable thru conventional non-AI approaches
– Short and long term
– Competitiveness
– Core business or new adjacent opportunities ?
– **As measured by (quantifiable KPI)?**

Cost estimation ($000) (ball park)
– Talent:
  – AI specialist
  – Product Mgr.
  – ML/SW Engineer
  – Data Engineer
  – SME

– The Client to cover:
  – Data
  – Infrastructure
  – Licenses

– Time: 12+ wks

Risk and challenges?
– What are undesired outcomes or potential problems that need to be addressed e.g. data quality, skills gap, security reviews
– Planned mitigations

Deliverable check-list review (1st group assignment)

1. Is the code clean and modular?
2. Can I someone external understand the code easily?
3. Does it use meaningful names?
4. Is there duplicated code?
5. Is there unused code?
6. Has code been optimized and refactored for production?
7. Is the code efficient?
8. Are there loops or other steps I can vectorize?
9. Can I use better data structures to optimize any steps?
10. Can I shorten the number of calculations needed for any steps?
11. Is the documentation effective?
12. Are inline comments concise and meaningful? i.e. Why not What
13. Is there complex code that's missing documentation?
14. Do functions use effective docstrings?
15. Is the necessary project documentation provided? i.e. README.md
16. Is the code well tested? i.e. each module has a test suit?
17. Does the code good test coverage?
18. Do tests check for interesting/ edge cases?
19. Are the tests readable?

Source: Udacity

ie UNIVERSITY

File Edit Selection View Go ··· mlops_project

EXPLORER

OPEN EDITORS
- main.py src 6

MLOPS_PROJECT
- data
- logs
  - main.log
- notebooks
- src
  - __pycache__
  - data_load
    - __pycache__
    - __init__.py
    - data_loader.py
  - __init__.py
  - main.py 6
- tests
- .env
- .gitignore
- config.yaml
- environment.yml
- README.md

OUTLINE

TIMELINE

main.py 6

src > main.py > setup_logging

```python
def setup_logging(logging_config: dict):
    Args:
        logging_config (dict): Logging configuration dictionary
    """
    log_file = logging_config.get("log_file", "logs/main.log")
    log_dir = os.path.dirname(log_file)
    if log_dir and not os.path.exists(log_dir):
        os.makedirs(log_dir, exist_ok=True)

    log_format = logging_config.get(
        "format", "%(asctime)s - %(levelname)s - %(name)s - %(message)s"
    )
    date_format = logging_config.get(
        "datefmt", "%Y-%m-%d %H:%M:%S"
    )
```

PROBLEMS 6 OUTPUT DEBUG CONSOLE ··· Filter (e.g. text, **/*.ts, !**/node_modules/**)

main.py src 6

- Using open without explicitly specifying an encoding Pylint(W1514:unspecified-encoding) [Ln 73, Col 10]
- Catching too general exception Exception Pylint(W0718:broad-exception-caught) [Ln 97, Col 12]
- Catching too general exception Exception Pylint(W0718:broad-exception-caught) [Ln 104, Col 12]
- Use lazy % formatting in logging functions Pylint(W1203:logging-fstring-interpolation) [Ln 120, Col 9]
- Catching too general exception Exception Pylint(W0718:broad-exception-caught) [Ln 122, Col 12]
- Use lazy % formatting in logging functions Pylint(W1203:logging-fstring-interpolation) [Ln 123, Col 9]

Check list 1st group assignment (I/II)

- 1 simple end-to-end "Data Science" Jupyter notebook ✅
  - Unique group (small) dataset
- Forked & Cloned the 'kick-off' repo ✅
- Group lead gave access to group members (collaborators) to the "Master" group's repo ✅
- Created and protected main & dev branches ✅
- Group lead broke linkage to professors' repo ☐
  - `git remote remove upstream`
- Each group member cloned own Master group's repo ✅
- Group defined and completed the Business case for project i.e. `README.md` ✅
- Downloaded (or cloned) the AI-assistant prompt scaffolder ✅

Check list 1st group assignment (II/II)

- Each collaborator was assigned ~2 'Modules'

```
│   ├── clean_data.py
│   ├── evaluate.py
│   ├── features.py
│   ├── infer.py
│   ├── load_data.py
│   ├── main.py
│   ├── train.py
│   ├── utils.py
│   └── validate.py
├── notebooks
│   ├── 00_opioid_analysis_vLegacy.ipynb
│   └── 01_opioid_analysis_vExp.ipynb
```

- Collaborators worked on their own branches e.g. *feature/inference*
  - *"Commit early, commit often, and push"*

- Collaborators created a comprehensive test suit for each of their modules

```
├── pytest.ini
└── tests
    ├── test_clean_data.py
```

- Both the Jupyter notebook (Experiments) and the main.py work end-to-end